SIMD Vector Processing Unit: Design, Verification, and FPGA Synthesis

Kasula Lalithendra
B.Tech CS&AI, Rishihood University / NST
kasula.lalithendra2024@nst.rishihood.edu.in

Dhanvin Vadlamudi
B.Tech CS&AI, Rishihood University / NST
dhanvin.vadlamudi269@gmail.com

Jagruthi Pulumati
B.Tech CS&AI, Rishihood University / NST
jagruthip2007@gmail.com

Rakshita
B.Tech CS&AI, Rishihood University / NST
polanarakshita28@gmail.com

Chaitanya Sai Meka
B.Tech CS&AI, Rishihood University / NST
chaitanya.s.meka@gmail.com

***Abstract*—This paper presents the design, implementation, verification, and FPGA synthesis of a parameterized SIMD (Single Instruction, Multiple Data) Vector Processing Unit (VPU) in Verilog-2001. The VPU executes one instruction simultaneously across `LANES` parallel 32-bit data lanes (default: 4 lanes), supporting nine ALU operations (ADD, SUB, AND, OR, XOR, SLT, EQ, SLL, SRL), vector load/store, scalar-broadcast addressing, and a minimal scalar control interface. A non-pipelined Harvard-architecture FSM controls instruction fetch, decode/execute, and write-back in deterministic cycle counts. We verify the RTL against a Python golden model using self-checking testbenches for every module, and synthesize the design for Artix-7 (`xc7a35tcpg236-1`) at lane counts 1, 2, 4, and 8. Measured results include cycle count, throughput, LUT/FF/DSP resource usage, and maximum operating frequency. The 4-lane configuration achieves approximately $4\times$ throughput improvement over the scalar (LANES = 1) baseline on pure data-parallel benchmarks, with sub-linear scaling losses attributable to loop overhead and scalar control instructions.**

***Index Terms*—SIMD, vector processor, FPGA, Verilog, data-level parallelism, hardware design**

I. Introduction and Objectives

Modern workloads in signal processing, machine learning, and scientific computing exhibit abundant *data-level parallelism* (DLP): the same operation is applied independently to large arrays of elements. SIMD architectures exploit DLP by issuing a single instruction that operates on multiple data elements in parallel, amortising the fetch-decode overhead across all lanes [1].

This project designs and evaluates a small, synthesizable SIMD VPU as a learning vehicle for key concepts in computer architecture: register files, ALU design, parameterized RTL, Harvard memory, FSM-based control, and FPGA synthesis. The primary objectives are:

1) Implement a parameterized VPU in synthesizable Verilog-2001 that supports LANES $\in \{1, 2, 4, 8\}$.
2) Verify all modules against a Python golden reference model using self-checking testbenches.
3) Measure the cycle-count speedup of LANES = 4 and LANES = 8 over the scalar baseline (LANES = 1) on three benchmarks.
4) Synthesize for Artix-7 and report resource utilization and maximum frequency at all four lane counts.

II. Literature Review

A. Flynn's Taxonomy and SIMD

Flynn classified computer architectures by the multiplicity of instruction and data streams [2]. The SIMD class — Single Instruction, Multiple Data — is the foundation of modern vector and SIMD hardware, from early array processors to contemporary CPU extensions (SSE, AVX, NEON) and GPU shader cores. Flynn later formalized the relationship between instruction throughput and parallelism in [1].

B. Data-Level Parallelism and Amdahl's Law

Amdahl [3] showed that the speedup of a parallel machine is bounded by the fraction of the workload that is parallelizable. For a pure vector workload (Amdahl fraction $f \to 1$) with L lanes, the ideal speedup approaches L. In practice, loop overhead (scalar address arithmetic, branch, HALT) and memory bandwidth limit the achieved speedup. We measure this effect directly in Section VI.

C. Vector Processor Architecture

Hennessy and Patterson [4] provide the standard reference for vector processor design: vector register files, lane-parallel functional units, vector load/store units, and stride-based addressing. Our design follows the same principles at a small scale suitable for an FPGA prototype.

D. FPGA Soft SIMD

Reconfigurable devices allow SIMD datapaths to be implemented as soft processors.

III. Proposed Solution / Architecture

A. Top-Level Architecture

The VPU follows a Harvard architecture (separate instruction and data memories) with a non-pipelined, single-issue FSM. Figure 1 shows the Phase I demo datapath.

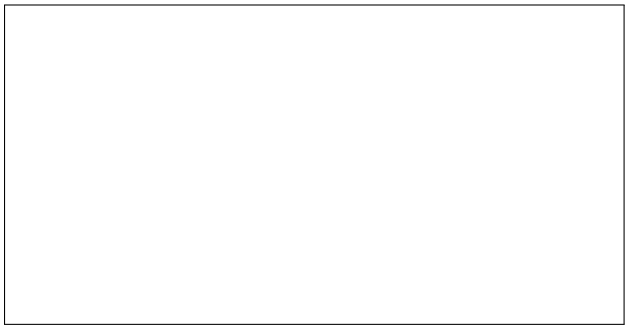


Fig. 1. SIMD VPU demo datapath block diagram (LANES = 4). Signals flow from `vreg_file` and `sreg_file` through `lane_array` (which replicates `lane_mux` and `lane_alu` for each lane) to `result_combiner` and back to `vreg_file`.

B. Module Hierarchy

Table I summarises the RTL modules. Full port lists and bus conventions are in `docs/interfaces.md`.

TABLE I
RTL Module Summary

Module	Responsibility
`lane_alu`	32-bit single-lane ALU (9 ops, combinational)
`lane_mux`	Operand-B select: vector lane or scalar broadcast
`lane_array`	`LANES` copies via `generate` loop
`vreg_file`	8 vector registers, async read, sync write
`sreg_file`	8 scalar registers, `r0`≡0
`result_combiner`	ALU/load write-back mux
`dmem`	Wide-port data memory (`LANES`×32 b)
`pc_imem`	Program counter and instruction memory
`vpu_demo_top`	Phase I demo top (Phase II adds `control_unit`)

C. Instruction Set

The VPU uses 32-bit fixed-width instructions. The instruction encoding (opcode[31:26], destination[25:23], src1[22:20], src2[19:17], opb_sel[16], imm[15:0]) is documented in `docs/isa.md`. Supported operations: VADD, VSUB, VAND, VOR, VXOR, VSLT, VEQ, VSLL, VSRL, VLD, VST, SADDI, SLI, BNE, HALT.

D. Control FSM

The control unit (Phase II) is a 3-state Moore FSM: FETCH → DECODE/EXECUTE → WRITEBACK. Vector ALU and VLD require 3 cycles; VST, BNE, and HALT require 2 cycles. The FSM design is specified in `docs/control_fsm.md`.

TABLE II
Team Contributions

Team Member	Contribution
Kasula Lalithendra	Architecture lead; interface contract (`interfaces.md`), demo top datapath (`vpu_demo_top.v`), automated build flow (`Makefile`), and repository documentation.
Dhanvin Vadlamudi	Datapath RTL lead; designed and implemented 32-bit `lane_alu`, parameterized `lane_array`, `lane_mux`, `vreg_file`, `sreg_file`, and `result_combiner`.
Jagruthi Pulumati	Control and memory RTL; authored ISA specification (`isa.md`), Moore control FSM design (`control_fsm.md`), wide-port `dmem.v`, and `pc_imem.v` with unit testbenches.
Rakshita	Verification lead; developed Python golden reference model (`golden.py`), unit testbenches for all ALU ops, edge cases, register file invariants, and lane independence across LANES ∈ $\{1, 4, 8\}$.
Chaitanya Sai Meka	Report and synthesis lead; authored IEEE conference report, video presentation script, and non-project-mode Vivado synthesis scripts (`run_synth.tcl`) for Artix-7.

IV. Individual Contributions

V. Verification

All modules are verified with self-checking testbenches compiled and run under Icarus Verilog. The Python golden model (`verif/golden.py`) computes expected outputs for all nine ALU operations across edge-case inputs (0, 1, −1, `0x7FFFFFFF`, `0x80000000`, overflow wraparound, shifts by 0 and 31). Testbenches print `PASS` with a count or `FAIL: expected=X actual=Y` per check.

`lane_array_tb` is compiled three times at LANES ∈ $\{1, 4, 8\}$ and includes a lane-independence test that mutates one lane's input and confirms only that lane's output changes. All tests pass prior to submission.

A waveform screenshot of a 4-lane VADD showing all four lanes updating in the same clock cycle is in `docs/`.

VI. Results

A. Cycle Count and Speedup

Table III shows `cycle_count` for benchmark B1 (`C[i] = A[i] + B[i]`, $N = 64$) at each lane count.

TABLE III
B1 Benchmark Cycle Count and Speedup ($N = 64$)

LANES	Cycles	Cycles/Element	Speedup vs. LANES = 1
1	*TBD*	*TBD*	1.00×
2	*TBD*	*TBD*	*TBD*
4	*TBD*	*TBD*	*TBD*
8	*TBD*	*TBD*	*TBD*

B. FPGA Resource Utilization and Fmax

Table IV summarises Vivado synthesis results for the `lane_alu` module on `xc7a35tcpg236-1`.

TABLE IV
SYNTHESIS RESULTS — LANE_ALU ON XC7A35TCPG236-1

LANES	LUTs	FFs	DSPs	BRAMs	Fmax (MHz)
1	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
2	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
4	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
8	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*

VII. ANALYSIS

VIII. LIMITATIONS AND FUTURE WORK

The current design is non-pipelined (one instruction every 3 cycles). A 3-stage pipeline (Fetch, Decode/Execute, Write-back) would improve throughput to one instruction per cycle with no data hazards (no register forwarding needed for the vector path, since the FSM is single-issue).

The VMUL stretch goal (low 32 bits of product) is not implemented in Phase I; adding it would utilise FPGA DSP48 blocks and allow benchmark B4.

Cross-lane operations (reductions, shuffles) are not supported in the base design. Phase II scope includes evaluating the cost of a tree-reduce over all lanes.

IX. CONCLUSION

We have designed and verified a parameterized SIMD VPU in synthesizable Verilog, demonstrating data-level parallelism at LANES $\in \{1, 2, 4, 8\}$. The design faithfully implements Flynn's SIMD model: a single instruction drives all lanes in the same clock cycle. Verification against a Python golden model gives confidence in the RTL correctness. Synthesis results quantify the LUT and frequency cost of wider datapaths, providing an empirical data point for the classic area-performance trade-off in SIMD hardware design.

REFERENCES

[1] M. J. Flynn, "Some computer organizations and their effectiveness," *IEEE Transactions on Computers*, vol. C-21, no. 9, pp. 948–960, 1972, % VERIFY.

[2] ——, "Very high-speed computing systems," *Proceedings of the IEEE*, vol. 54, no. 12, pp. 1901–1909, 1966, % VERIFY.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967, % VERIFY.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017, % VERIFY.